1851. Minimum Interval to Include Each Query

Hard · Intervals

leetcode.com/problems/minimum-interval-to-include-each-query

Problem

- For each query q, return the size (right - left + 1) of the smallest interval containing q, or -1.

Intuition

- Process queries in increasing order (offline). As q grows, add every interval whose left <= q into a min-heap keyed by size.
- Pop intervals whose right < q: they can never contain this query or any later one. The heap top is the answer.

Brute Force

- For each query scan all intervals: O(n q).

Optimal Approach

- Sort intervals by left; sort query indices by value.
- For each query q (ascending): push all intervals with left <= q as (size, right).
- Pop while top.right < q. Answer = top.size, or -1 if the heap is empty.

Worked Example

```
intervals [[1,4],[2,4],[3,6],[4,4]], queries [2,3,4,5].

q=2: push [1,4](4) [2,4](3)          top size 3
q=3: push [3,6](4)                   top size 3
q=4: push [4,4](1)                   top size 1
q=5: pop [4,4],[2,4],[1,4] (right<5)  top [3,6] size 4
answer [3,3,1,4]
```

Java Solution

```java
public class MinimumIntervalToIncludeEachQuery {
    public int[] minInterval(int[][] intervals, int[] queries) {
        int[][] iv = intervals.clone();
        Arrays.sort(iv, (a, b) -> Integer.compare(a[0], b[0]));
        Integer[] order = new Integer[queries.length];
        for (int i = 0; i < order.length; i++) order[i] = i;
        Arrays.sort(order, (a, b) -> Integer.compare(queries[a], queries[b]));
        PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> Integer.compare(a[0], b[0])); // {size, right}
        int[] ans = new int[queries.length];
        int i = 0;
        for (int qi : order) {
            int q = queries[qi];
            while (i < iv.length && iv[i][0] <= q) { pq.add(new int[]{iv[i][1] - iv[i][0] + 1, iv[i][1]}); i++; }
            while (!pq.isEmpty() && pq.peek()[1] < q) pq.poll();
            ans[qi] = pq.isEmpty() ? -1 : pq.peek()[0];
        }
        return ans;
    }
}
```

Complexity

- Time: $O((n + q) \log n + q \log q)$. Space: $O(n + q)$.

Pitfalls

- Keep the original query positions when sorting so answers go back in order.